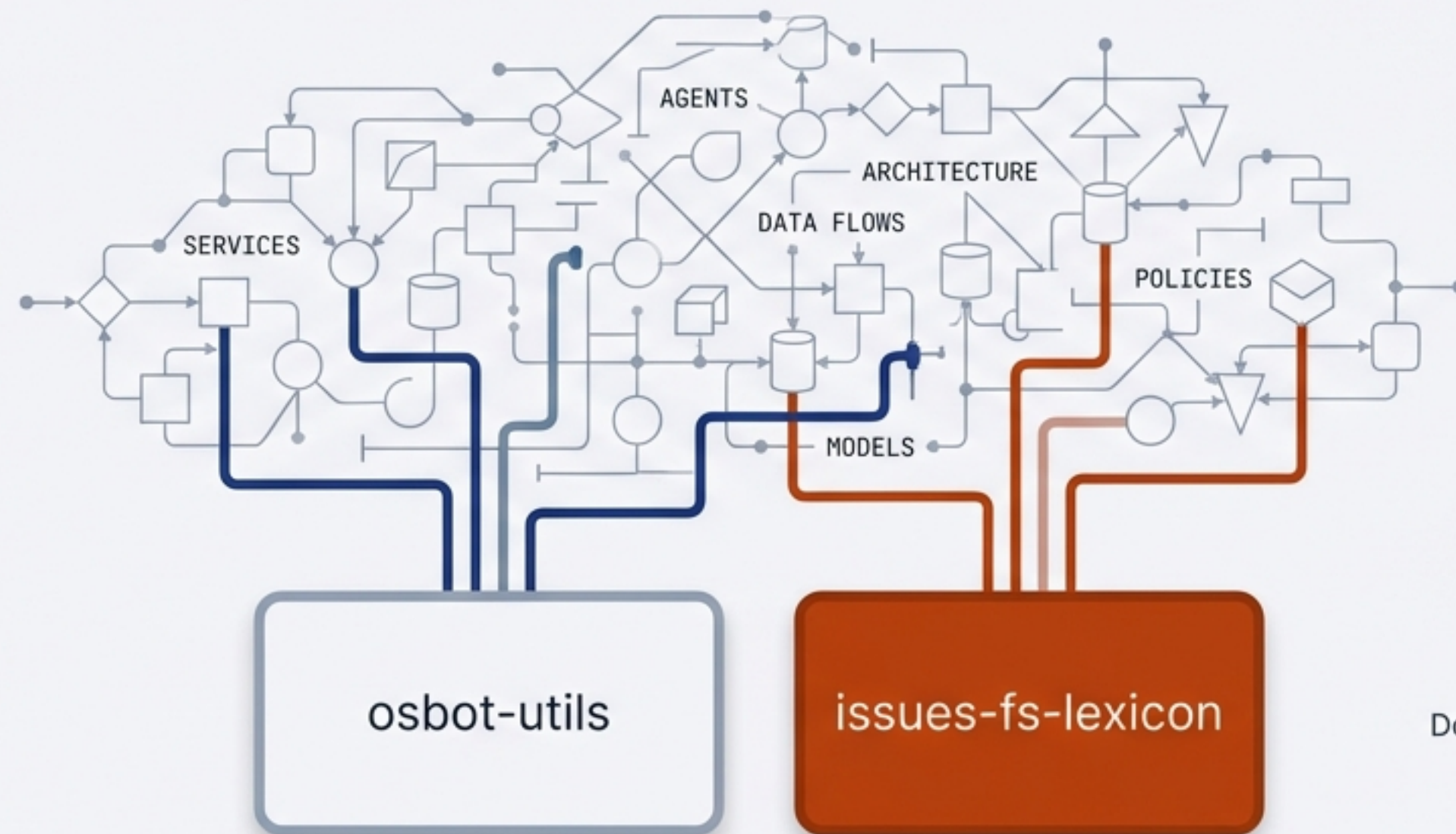


Issues-FS Lexicon: The Common Language of the Ecosystem

Standardising the Semantic Foundation for
Architecture, Services, and Agents



Document: issues-fs_lexicon-architecture
Version: v1.0 (Draft)
Date: 05 February 2026

The Architectural Invariant: Separating Code from Meaning

The Issues-FS architecture relies on two universal dependencies that every repository must consume. This separation of concerns ensures that code mechanics and domain semantics evolve independently but function coherently.

- `osbot-utils`: Defines how we write code (Type_Safe utilities).
- `issues-fs-lexicon`: Defines what we talk about (Ontologies, Taxonomies, Schemas).

The Definition

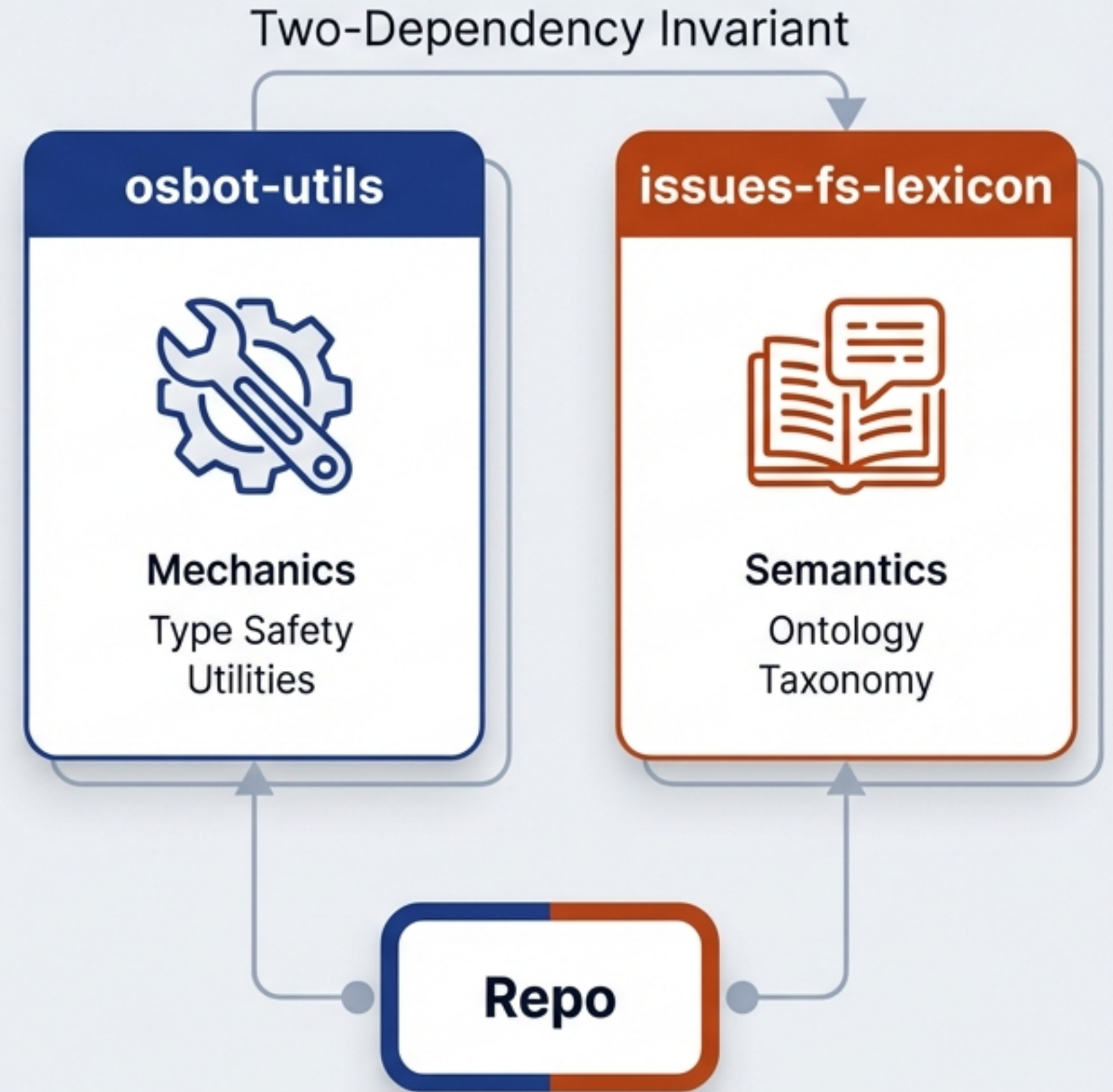
The Lexicon is a PyPI package (`issues-fs-lexicon`) containing shared schemas, domain definitions, and coordination protocols.

Key Benefits

Universal Reference: Any repo can reference shared concepts without heavy imports.

Clean Propagation: Vocabulary changes happen in one place and propagate on version bumps.

Zero Bootstrapping: New repos possess a shared vocabulary from line one.

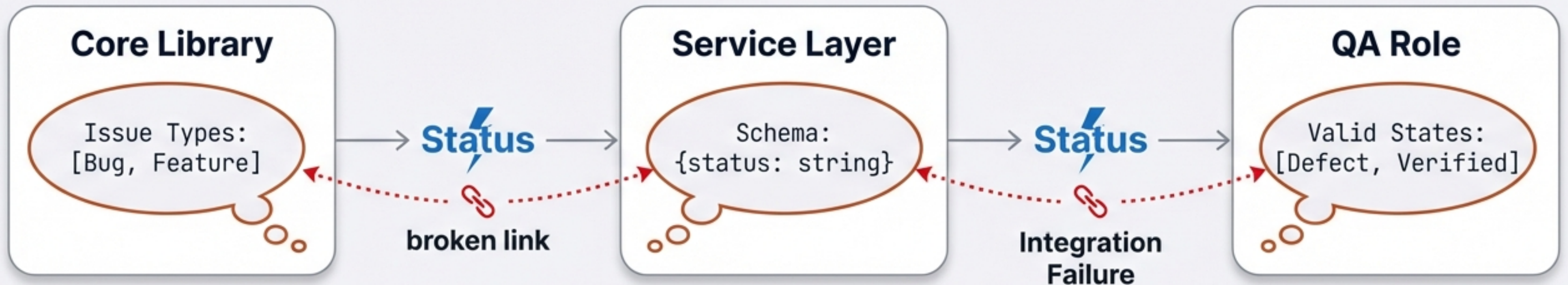


The Drift: Why Every Ecosystem Needs a Dictionary

As ecosystems scale from Core to Service to Integrations and Roles, definitions inevitably diverge. This “Tower of Babel” effect creates invisible friction where different components hold different truths about the same concepts.

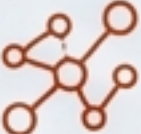
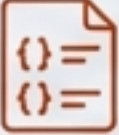
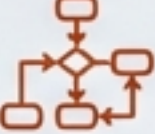
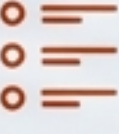
The Consequence: Subtle bugs, schema mismatches, broken integrations, and agents talking past each other.

Vocabulary Drift



Beyond Schemas: Defining the Complete Vocabulary

A 'Schemas' repo is too narrow. The Lexicon encompasses the terms, definitions, relationships, structures, and behavioral rules of the domain.

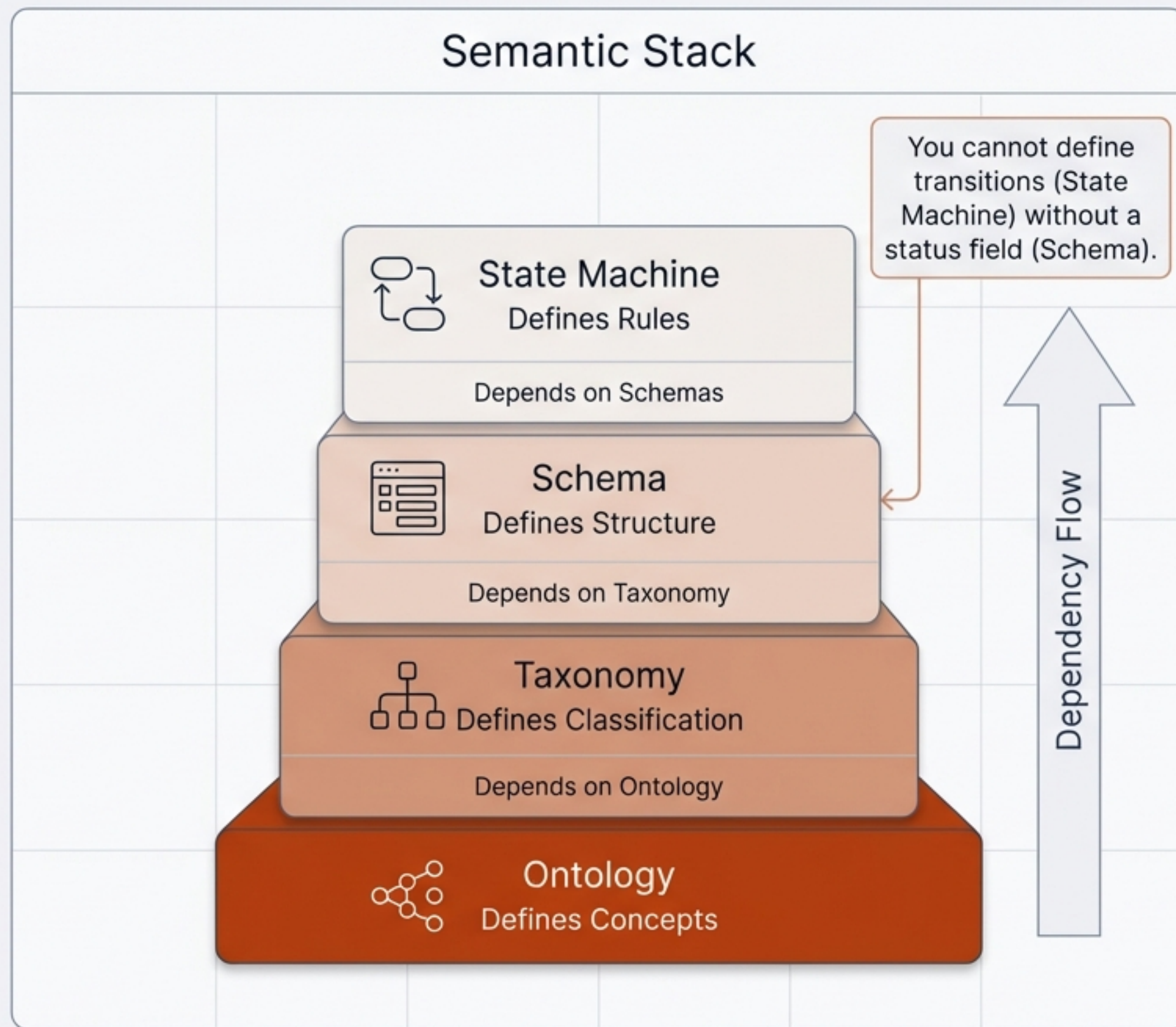
Artifact Inventory			
Icon	Artifact Name	Description	Example
	Ontology	Concepts and relationships	"A Decision is a type of Issue"
	Taxonomy	Classification hierarchies	"issue-types.json, link-types.json"
	Schema	Structure of concepts	"Decision issue fields"
	State Machine	Behavioural rules	"Valid status transitions"
	Vocabulary	Definitions of terms	"Status, Priority, Severity"
	Base Classes	Shared code encoding vocabulary	"Type_Safe classes"
	Constants	Standard field names and enums	"Enum definitions to prevent drift"

The Semantic Stack

A Layered Model of Meaning

The artifacts are not a flat list but a hierarchy of dependencies. Higher layers rely on the context of lower layers to function.

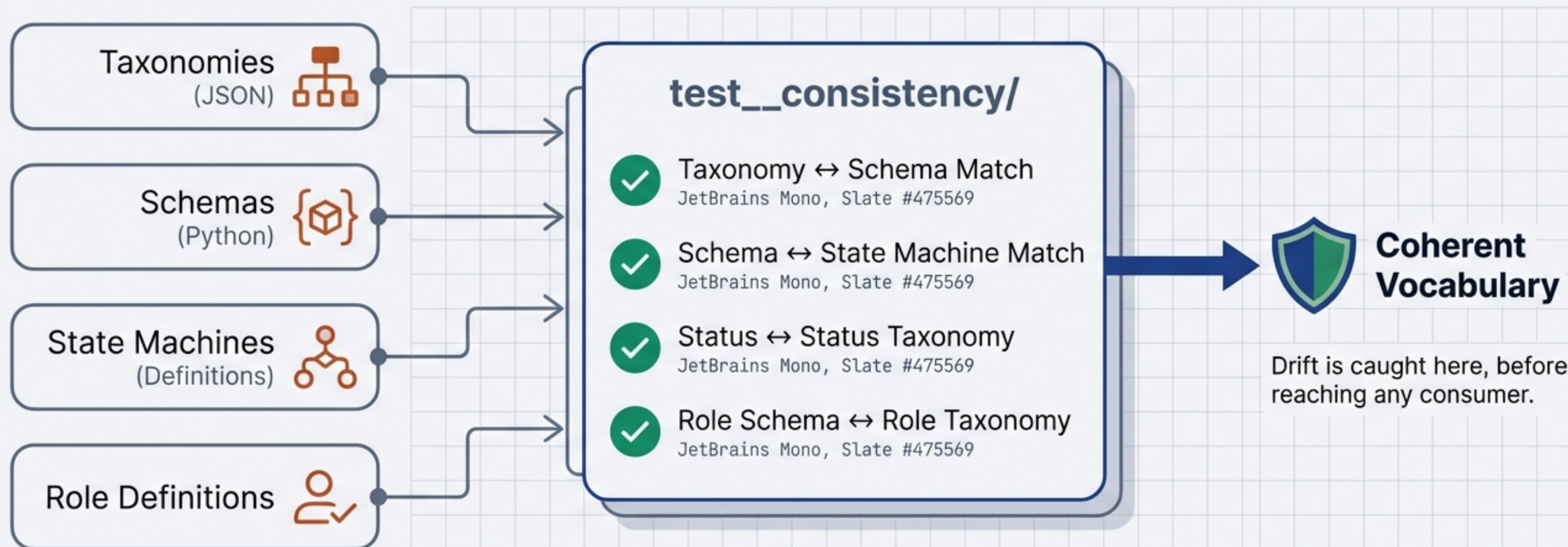
The Lexicon owns all four layers, ensuring the stack remains coherent.



The Guardian of Truth: Automated Consistency

Preventing drift at the source via the “Librarian” tests.

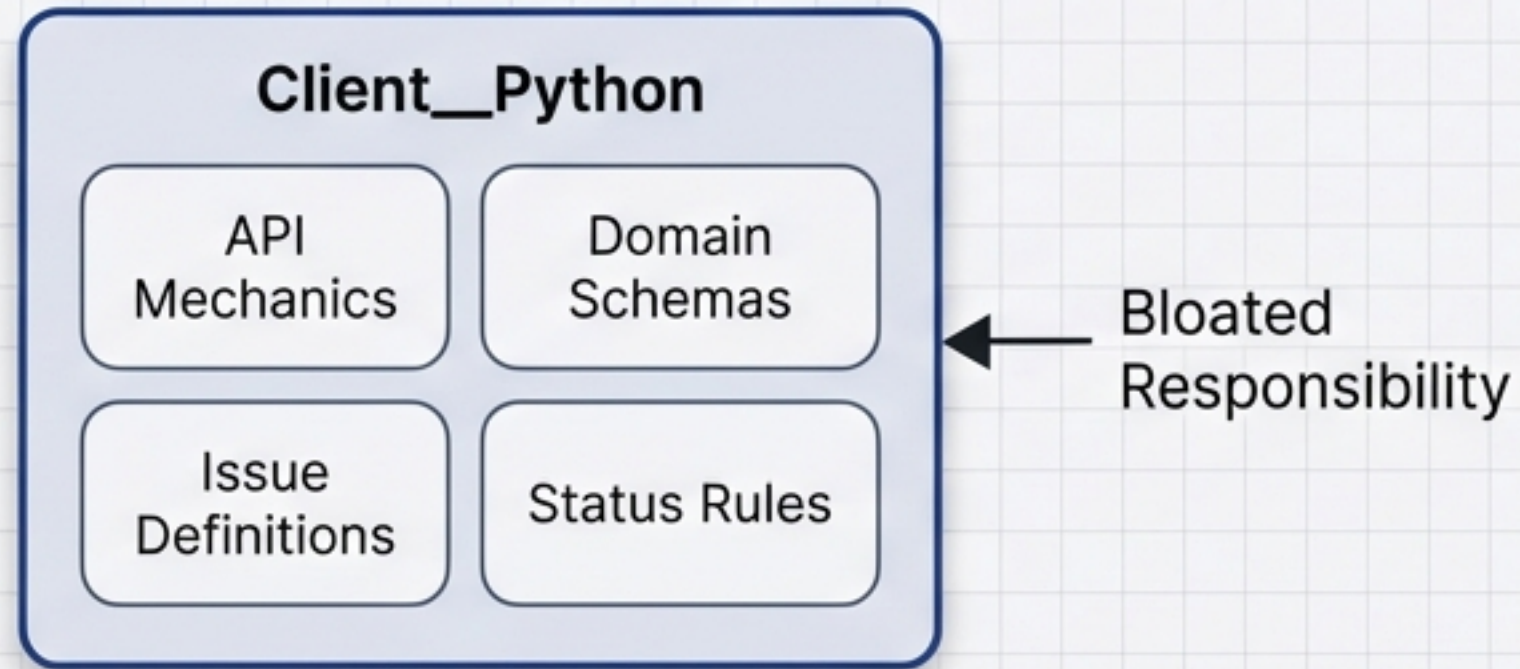
The “test_consistency/” directory acts as the automated Librarian. It ensures that MGraph representations remain in sync with Python definitions.



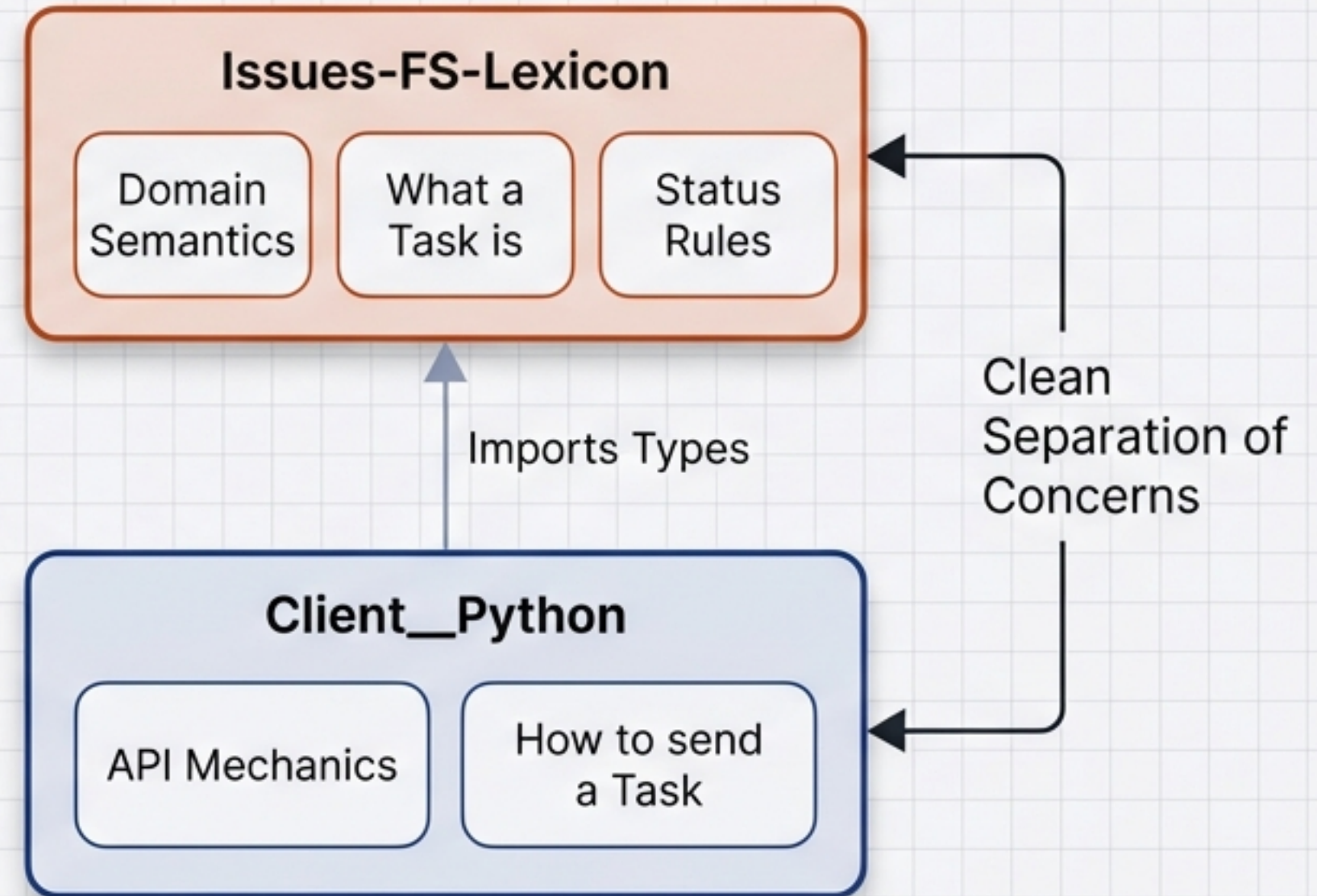
The Dependency Shift: Refactoring Responsibility

Decoupling API mechanics from Domain semantics.

Before

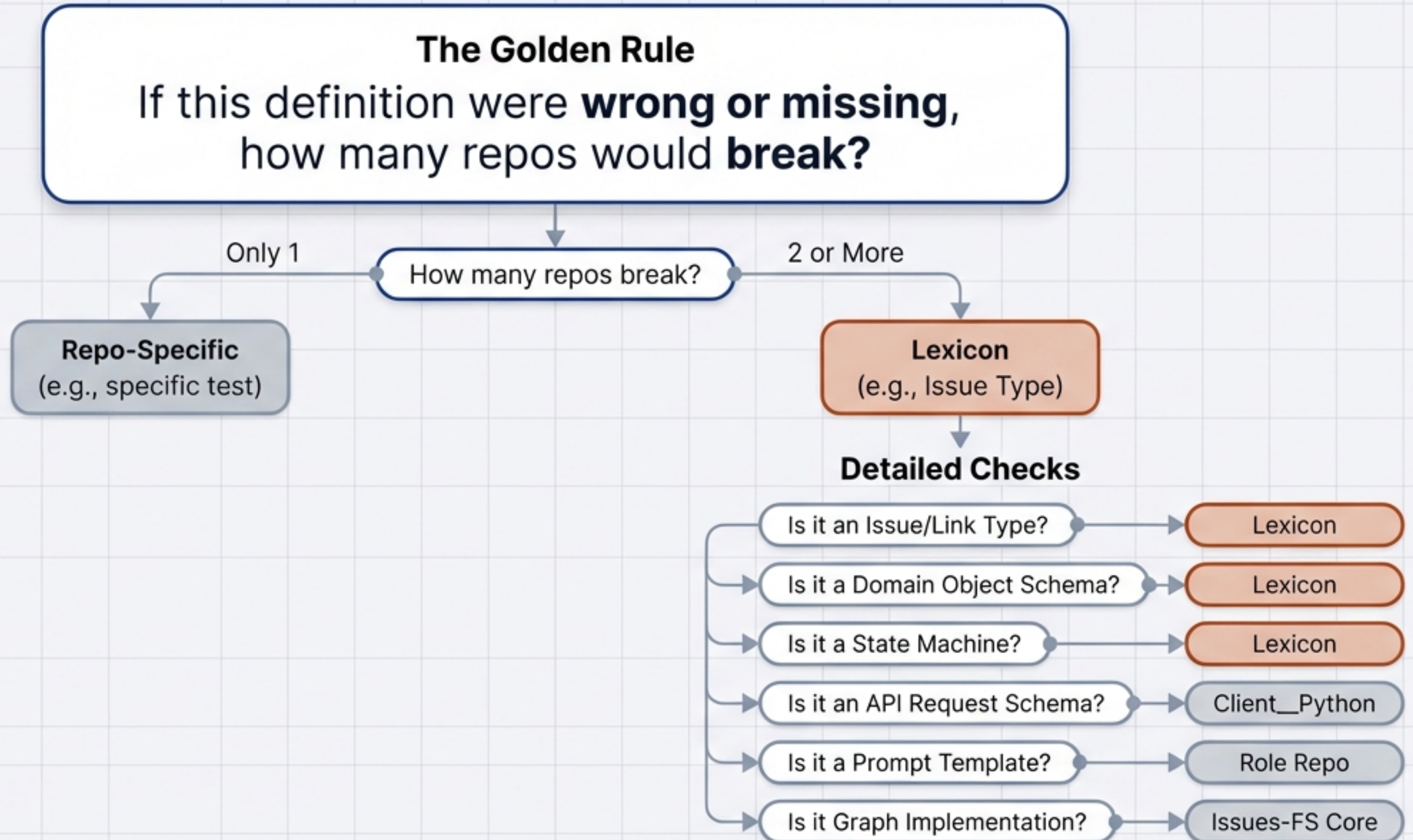


After



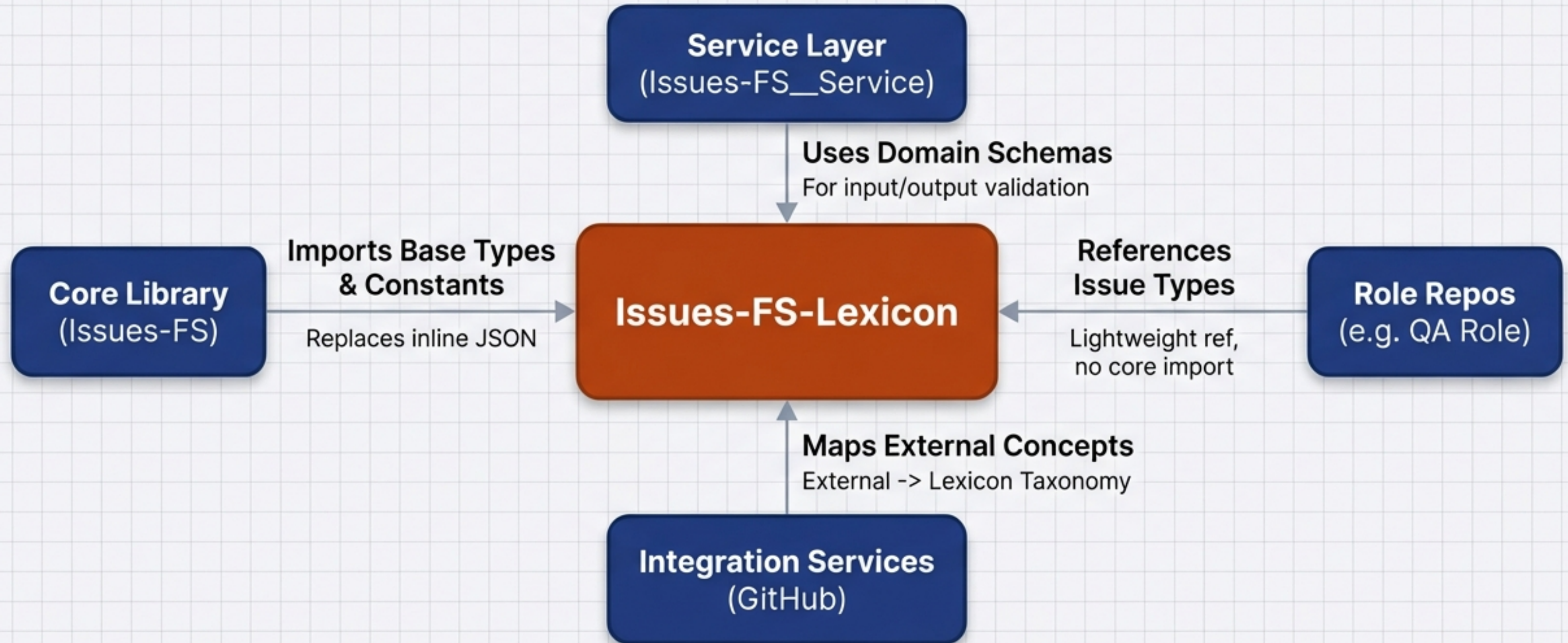
The Litmus Test: What Belongs in the Lexicon?

Deciding where definitions live based on impact and reusability.

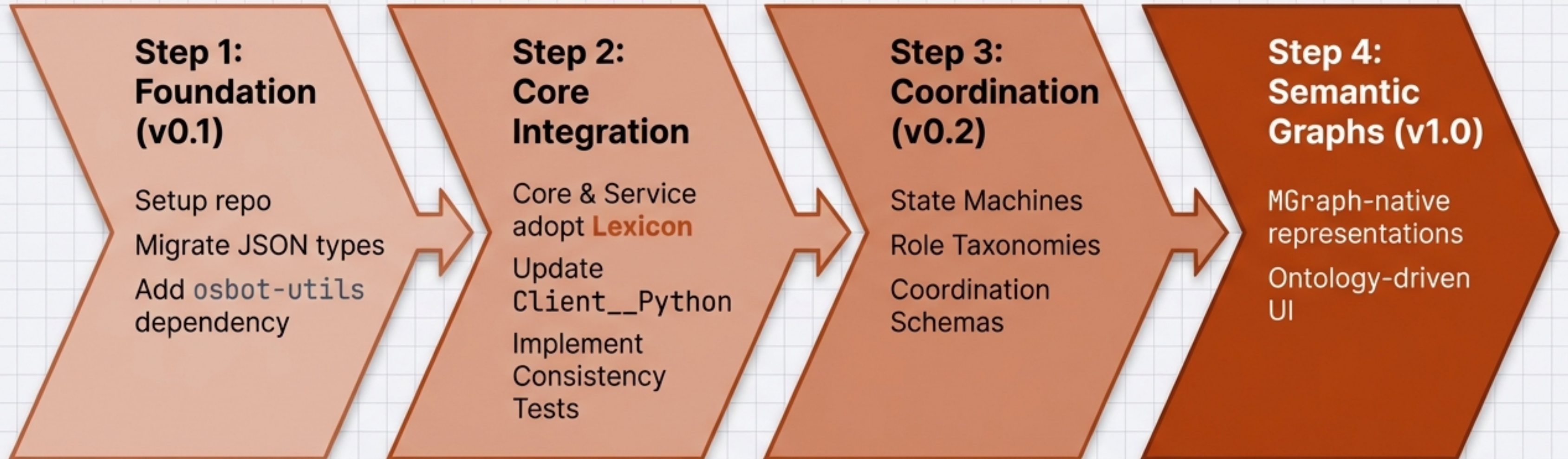


Integration Patterns: One Source, Many Consumers

How different components consume and leverage the centralized Lexicon.

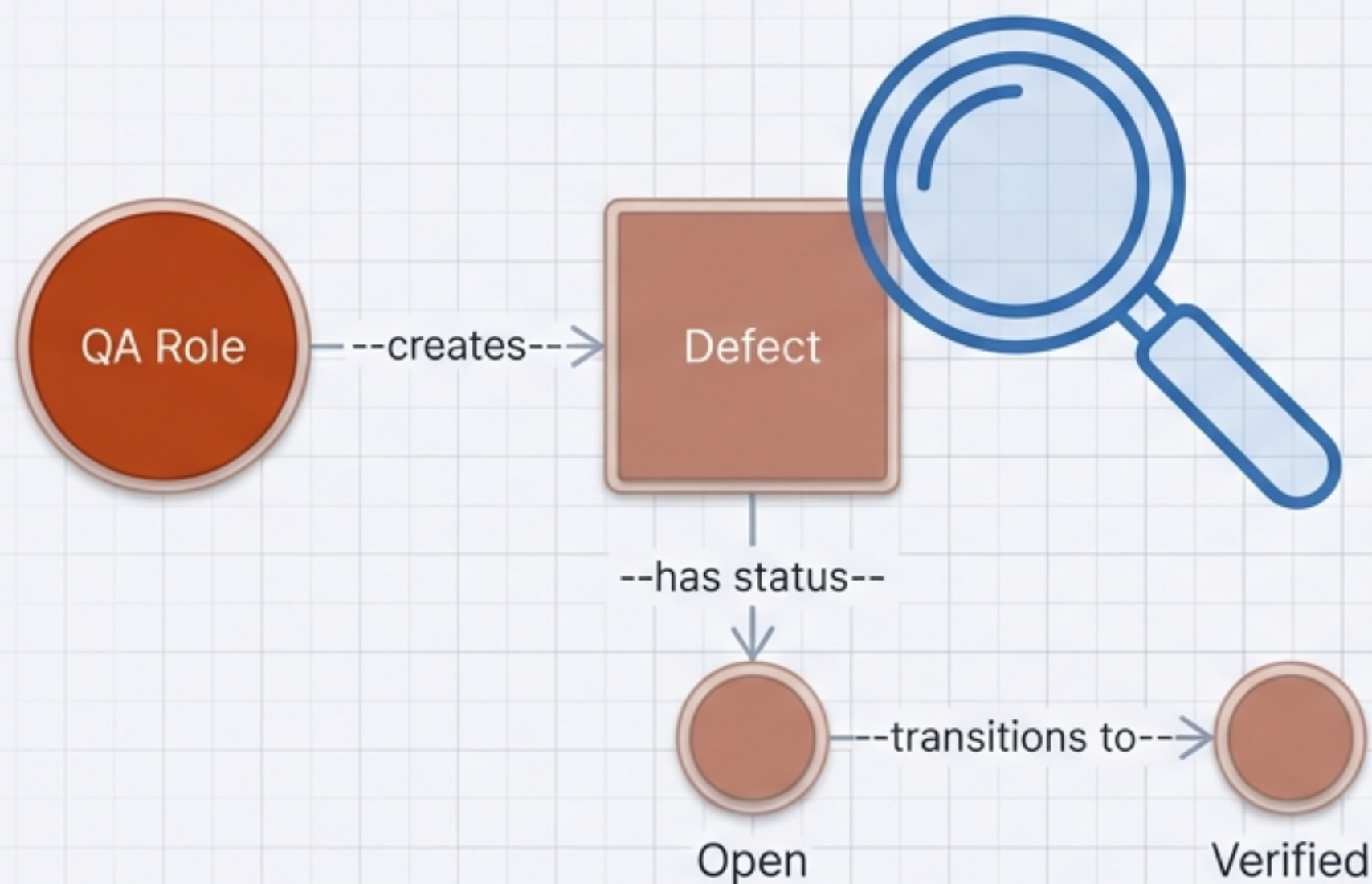


The Migration Path: Incremental Adoption



The Future: From Static Files to Semantic Graph

Moving from Python classes to a runtime queryable brain.



- **Graph Queries:** "Show me all `issue` types a `QA role` can create."
- **Schema Comparison:** "What fields do `Decision` and `Handoff` share?"
- **Path Analysis:** "Can `Defect` transition to `Verified` directly?"

Architectural Decisions Log

Rationale behind the key structural choices.

ID	Decision	Rationale
L1	Name: Lexicon	Signals complete vocabulary (docs + rules), not just schemas.
L2	Universal Dependency	Prevents import friction and ensures consistency.
L3	Separation of Concerns	Domain schemas in Lexicon ; API schemas in Client.
L4	Data Migration	Vocabulary JSONs belong in the vocabulary repo.
L5	Consistency Tests	Validation lives where the definitions are.

Conclusion: Enabling Agent Coordination

By standardising our language, we move from fragile integrations to robust coordination.

From:

Implicit definitions
and vocabulary drift.

To:

A shared, type-safe,
queryable foundation.



Inter – We enable our agents and services to collaborate without confusion.

📌 Adopt `issues-fs-lexicon v0.1`

References & Dependencies

Documentation

- Issues-FS Architecture Overview
 - Issues-FS Role-Based Agent Coordination
 - Issues-FS Role Architecture Framework Analysis
-

Technical Dependencies

- osbot-utils: The companion universal dependency (Type_Safe utilities).
- MGraph-DB: The future storage layer for the Lexicon graph.